

1 IMPLEMENT DDL, DML COMMANDS

AIM:

To implement the Data Definition Language (DDL) and Data Manipulation Language (DML) commands for a university database using PL/SQL. This program will create tables with key constraints, insert sample records, and demonstrate a query to retrieve related data.

ALGORITHM:

1. Create the essential tables: Departments, Instructors, Courses, Students, and Enrollments with appropriate primary and foreign key constraints.
2. Insert sample data records into each table.
3. Commit the transactions to save the changes.
4. Write a sample join query to retrieve student names along with their enrolled course names.

PROGRAM:

```
-- DDL Commands: Create tables for University Database

CREATE TABLE Departments (
    dept_id NUMBER PRIMARY KEY,
    dept_name VARCHAR2(50) NOT NULL,
    location VARCHAR2(100)
);

CREATE TABLE Instructors (
    instructor_id NUMBER PRIMARY KEY, name
    VARCHAR2(100) NOT NULL,
    dept_id NUMBER,
    CONSTRAINT fk_dept_inst FOREIGN KEY (dept_id) REFERENCES
Departments(dept_id)
);

CREATE TABLE Courses (
    course_id NUMBER PRIMARY KEY,
    course_name VARCHAR2(100) NOT NULL,
    credits NUMBER,
    dept_id NUMBER,
    CONSTRAINT fk_dept_course FOREIGN KEY (dept_id) REFERENCES
Departments(dept_id)
);

CREATE TABLE Students (
    student_id NUMBER PRIMARY KEY,
```

```
Departments(dept_id)
);

CREATE TABLE Enrollments (
    enrollment_id NUMBER PRIMARY KEY,
    student_id NUMBER,
    course_id NUMBER,
    semester VARCHAR2(50),
    CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES Students(student_id)
    CONSTRAINT fk_course FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

-- DML Commands: Insert sample records
-- Departments
INSERT INTO Departments VALUES (1, 'Computer Science', 'Building A'); INSERT INTO
Departments VALUES (2, 'Physics', 'Building B');
INSERT INTO Departments VALUES (3, 'Mathematics', 'Building C');

-- Instructors
INSERT INTO Instructors VALUES (101, 'Dr. Alan Turing', 1); INSERT INTO
Instructors VALUES (102, 'Dr. Marie Curie', 2); INSERT INTO Instructors
VALUES (103, 'Dr. Isaac Newton', 3);

-- Courses
INSERT INTO Courses VALUES (201, 'Database Systems', 4, 1);
INSERT INTO Courses VALUES (202, 'Quantum Physics', 3, 2);
INSERT INTO Courses VALUES (203, 'Calculus', 4, 3);

-- Students
INSERT INTO Students VALUES (301, 'Alice Smith',
TO_DATE('2002-04-15', 'YYYY-MM-DD'), 1);
INSERT INTO Students VALUES (302, 'Bob Johnson',
TO_DATE('2001-09-23', 'YYYY-MM-DD'), 2);
INSERT INTO Students VALUES (303, 'Clara Oswald',
TO_DATE('2003-01-10', 'YYYY-MM-DD'), 3);

-- Enrollments
```

```
INSERT INTO Enrollments VALUES (401, 301, 201, 'Fall 2025');
INSERT INTO Enrollments VALUES (402, 302, 202, 'Spring 2025');
INSERT INTO Enrollments VALUES (403, 303, 203, 'Fall 2025');

-- Commit Transactions
COMMIT;

-- Sample Query: Select students and their courses
SELECT s.name AS Student_Name, c.course_name AS Course_Name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id JOIN
```

OUTPUT:

Student_Name	Course_Name
Alice Smith	Database Systems
Bob Johnson	Quantum Physics
Clara Oswald	Calculus

RESULT:

This output indicates successful execution of **DDL** (CREATE, ALTER, DROP) and **DML** (INSERT, UPDATE, DELETE) commands in PL/SQL.

2 PL/SQL: CASE, LOOP

AIM:

To demonstrate the use of PL/SQL CASE statements and loops by processing student grades and displaying messages based on their performance. The program will iterate over student records, assign grade categories using CASE, and display the results.

ALGORITHM:

1. Create a temporary table or use an existing table with student grades data.
2. Use a cursor or loop to iterate over each student's grade record.
3. Use a CASE statement inside the loop to classify grades into categories (e.g., Excellent, Good, Average, Poor).
4. Print the student's name, grade, and category using DBMS_OUTPUT.
5. End the loop after processing all rows.

PROGRAM:

```
SET SERVEROUTPUT ON;
-- Create Students_Grades table
CREATE TABLE Students_Grades (
    student_id NUMBER PRIMARY KEY,
    name VARCHAR2(100) NOT NULL,
    grade NUMBER CHECK (grade BETWEEN 0 AND 100)
);

-- Insert sample records
INSERT INTO Students_Grades VALUES (301, 'Alice Smith', 92); INSERT
INTO Students_Grades VALUES (302, 'Bob Johnson', 78); INSERT INTO
Students_Grades VALUES (303, 'Clara Oswald', 65);
```

```

-- Commit the changes
COMMIT;

DECLARE
    CURSOR cur_student_grades IS
        SELECT student_id, name, grade FROM Students_Grades; -- Assume this table
        exists

    v_name Students_Grades.name%TYPE;
    v_grade Students_Grades.grade%TYPE;
    v_category VARCHAR2(20);
BEGIN
    FOR rec IN cur_student_grades LOOP
        v_name := rec.name;
        v_grade := rec.grade;

        -- Using CASE to categorize grades
        v_category := CASE
            WHEN v_grade >= 90 THEN 'Excellent' WHEN
            v_grade >= 75 THEN 'Good'
            WHEN v_grade >= 60 THEN 'Average'
            ELSE 'Poor'
        END;

        DBMS_OUTPUT.PUT_LINE('Student: ' || v_name || ', Grade: ' || v_grade || ', Category: ' || v_category);
    END LOOP;
END;

```

OUTPUT:

```

text
Student: Alice Smith, Grade: 92, Category: Excellent
Student: Bob Johnson, Grade: 78, Category: Good
Student: Clara Oswald, Grade: 65, Category: Average
Student: David Tennant, Grade: 55, Category: Poor

```

RESULT:

Thus, This program illustrates control structures and decision making in PL/SQL through CASE and loop usage on student grade data.

3 IMPLEMENT FOURTH AND FIFTH FORM OF NORMALIZATION TECHNIQUES

AIM:

To demonstrate Fourth Normal Form (4NF) and Fifth Normal Form (5NF) by creating fresh tables representing multi-valued dependencies and join dependencies, then decomposing them to eliminate redundancies and anomalies according to these advanced normalization rules.

ALGORITHM:

1. Start with a table exhibiting multi-valued dependencies for 4NF demonstration, e.g., a **Student_Activities** table with students and multiple independent activities (clubs and sports).
2. Normalize to 4NF by splitting into two tables where no table has more than one multi-valued dependency.
3. For 5NF, start with a table that can be decomposed into multiple joinable tables without loss, e.g., **Project_Assignments** showing assignments of employees to projects with specific roles.
4. Decompose it into smaller tables to remove join dependencies and restore by joining only.
5. Insert sample data at each stage to illustrate the process.

PROGRAM:

We start with a table having multi-valued dependencies and normalize it into two separate tables.

```
-- Unnormalized table with multi-valued dependencies CREATE
TABLE Student_Activities (
  student_id NUMBER,
  student_name VARCHAR2(100), club
  VARCHAR2(50),
  sport VARCHAR2(50),
  PRIMARY KEY (student_id, club, sport)
);

-- Insert sample data into Student_Activities
INSERT INTO Student_Activities VALUES (1, 'Alice Smith', 'Chess Club',
```



```

INSERT INTO Student_Activities      VALUE (1, 'Alice Smith', 'Drama
Club', 'Basketball');
INSERT INTO Student_Activities      VALUE (1, 'Alice Smith', 'Chess
Club', 'Soccer');
INSERT INTO Student_Activities      VALUE (2, 'Bob Johnson', 'Chess
Club', 'Tennis');
INSERT INTO Student_Activities      VALUE (2, 'Bob Johnson', 'Drama
Club', 'Tennis');

COMMIT;

-- Normalize into 4NF by splitting into two tables

-- Table for Student Clubs
CREATE TABLE Student_Clubs
( student_id NUMBER,
  student_name VARCHAR2(100), club
  VARCHAR2(50),
  PRIMARY KEY(student_id, club)
);

-- Table for Student Sports
CREATE TABLE Student_Sports ( student_id
  NUMBER,
  student_name VARCHAR2(100), sport
  VARCHAR2(50),
  PRIMARY KEY(student_id, sport)
);

-- Insert data extracted from original table to normalized tables
INSERT INTO Student_Clubs
(student_id, student_name, club)
SELECT DISTINCT student_id, student_name, club FROM Student_Activities;

INSERT INTO Student_Sports (student_id, student_name, sport) SELECT
DISTINCT student_id, student_name, sport FROM
Student_Activities; COMMIT;

```

Fifth Normal Form (5NF) Example:

Decompose a table with join dependencies into smaller tables.

```
-- Table with join dependencies
CREATE TABLE Project_Assignments
( employee_id NUMBER,
  project_id NUMBER, role
  VARCHAR2(50),
  PRIMARY KEY (employee_id, project_id, role)
);

-- Insert sample data
INSERT INTO Project_Assignments VALUES (101, 201, 'Developer'); INSERT
INTO Project_Assignments VALUES (101, 202, 'Tester');
INSERT INTO Project_Assignments VALUES (102, 201, 'Developer'); INSERT
INTO Project_Assignments VALUES (103, 203, 'Manager');

COMMIT;

-- Decompose into three tables to eliminate join dependencies CREATE
TABLE Employee_Projects (
  employee_id NUMBER,
  project_id NUMBER,
  PRIMARY KEY (employee_id, project_id)
);

CREATE TABLE Employee_Roles ( employee_id
  NUMBER,
  role VARCHAR2(50),
  PRIMARY KEY (employee_id, role)
);
```

```
PRIMARY KEY (project_id, role)
);
```

```
-- Insert decomposed data from Project_Assignments
INSERT INTO Employee_Projects (employee_id, project_id)
SELECT DISTINCT employee_id, project_id FROM Project_Assignments;

INSERT INTO Employee_Roles (employee_id, role)
SELECT DISTINCT employee_id, role FROM Project_Assignments;

INSERT INTO Project_Roles (project_id, role)
SELECT DISTINCT project_id, role FROM Project_Assignments;
```

Verification Queries (to display normalized data):

```
-- 4NF Tables
SELECT * FROM Student_Clubs ORDER BY student_id, club; SELECT *
FROM Student_Sports ORDER BY student_id, sport;

-- 5NF Tables
SELECT * FROM Employee_Projects ORDER BY employee_id, project_id; SELECT
* FROM Employee_Roles ORDER BY employee_id, role;
```

OUTPUT:

Output for 4NF Tables

Student_Clubs:

student_id	student_name	club
1	Alice Smith	Chess Club
1	Alice Smith	Drama Club
2	Bob Johnson	Chess Club
2	Bob Johnson	Drama Club

Student_Sports:

student_id	student_name	sport
1	Alice Smith	Basketball
1	Alice Smith	Soccer
2	Bob Johnson	Tennis

Output for 5NF Tables

Employee_Projects:

employee_id	project_id
101	201
101	202
102	201
103	203

Employee_Roles:

employee_id	role
101	Developer
101	Tester
102	Developer
103	Manager

Project_Roles:

project_id	role
201	Developer
202	Tester
203	Manager

RESULT:

This full script shows creating tables from scratch, populating with sample data, performing normalization to 4NF and 5NF forms, and storing decomposed data, thereby eliminating multi-valued and join dependencies while maintaining all information through decomposition

4 IMPLEMENT INTEGRITY CONSTRAINTS

AIM:

To create a fresh set of tables in a university database that implement various integrity constraints such as primary keys, foreign keys, unique constraints, not null constraints, and check constraints. The program will insert valid and invalid data to demonstrate constraint enforcement.

ALGORITHM:

1. Define tables **Departments**, **Professors**, and **Courses** with integrity constraints.
2. Apply primary key constraints on ID columns for uniqueness.
3. Apply foreign key constraints to link **Courses** to **Departments** and **Professors**.
4. Use NOT NULL and UNIQUE constraints for essential fields.
5. Use CHECK constraints to validate domain-specific rules (e.g., course credits positive).
6. Insert sample valid records to demonstrate successful constraint enforcement.
7. Attempt to insert invalid records (commented out or in explanation) to show constraint violations.

PROGRAM:

```
-- Create Departments table with NOT NULL and UNIQUE constraints CREATE
TABLE Departments (
    dept_id NUMBER PRIMARY KEY,
    dept_name VARCHAR2(50) NOT NULL UNIQUE
);

-- Create Professors table with constraints CREATE
TABLE Professors (
    professor_id NUMBER PRIMARY KEY,
    professor_name VARCHAR2(100) NOT NULL,
    dept_id NUMBER NOT NULL,
    CONSTRAINT fk_prof_dept FOREIGN KEY (dept_id) REFERENCES
Departments(dept_id)
);

-- Create Courses table with check constraints and foreign keys CREATE
TABLE Courses (
    course_id NUMBER PRIMARY KEY,
    course_name VARCHAR2(100) NOT NULL UNIQUE,
    credits NUMBER NOT NULL CHECK (credits > 0), dept_id
NUMBER NOT NULL,
    professor_id NUMBER NOT NULL,
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id) REFERENCES
Departments(dept_id),
    CONSTRAINT fk_course_prof FOREIGN KEY (professor_id) REFERENCES
Professors(professor_id)
```

Expected Output:

- Successful creation of all tables with constraints.
- Successful insertion of valid records.
- On attempting invalid inserts (if uncommented), the database throws errors such as UNIQUE constraint violation, CHECK constraint violation, or foreign key violation.

This program enforces data integrity at multiple levels, ensuring consistent and reliable data in the university database environment.

RESULT:

This program enforces data integrity at multiple levels, ensuring consistent and reliable data in the university database environment.